CRYPTO LAB-7

NAME: ABISHEK K G **REG NO: 23BCE1739**
DATE: 07/03/2026 **SLOT: L29+L30**

Qn.) Develop a client-server application where the client accepts a user-input message, converts it into its binary equivalent, and applies SHA-512 preprocessing, including padding and message length encoding, to form a 1024-bit message block. The client extracts the first 16 words (WO to W15) of the message schedule and sends them to the server. The server receives the message schedule words from the client and displays the values for verification. Implement this using socket programming in any language and verify the output.

Code:

```
import java.io.*;

import java.net.*;

import java.util.*;


public class SHA512Socket {

   public static void main(String[] a) throws Exception {

      if (a.length > 0 && a[0].equals("s")) {

         ServerSocket ss = new ServerSocket(5000);

         System.out.println("server waiting...");

         Socket s = ss.accept();

         DataInputStream in = new DataInputStream(s.getInputStream());

         System.out.println("received message schedule words:");

         for (int i = 0; i < 16; i++)

            System.out.printf("W%-2d = %016X%n", i, in.readLong());

         in.close(); s.close(); ss.close();

      } else {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("enter message: ");
        String msg = sc.nextLine();
        byte[] mb = msg.getBytes("UTF-8");
        int ml = mb.length;
        System.out.println("binary representation:");
        for (byte b : mb)
            System.out.print(String.format("%8s", Integer.toBinaryString(b & 0xFF)).replace(' ', '0')
+ " ");
        System.out.println();
        int pl = ((ml + 17 + 127) / 128) * 128;
        byte[] pb = new byte[pl];
        System.arraycopy(mb, 0, pb, 0, ml);
        pb[ml] = (byte) 0x80;
        long bl = (long) ml * 8;
        for (int i = 0; i < 8; i++)
            pb[pl - 1 - i] = (byte) (bl >>> (i * 8));
        long[] w = new long[16];
        for (int i = 0; i < 16; i++)
            for (int j = 0; j < 8; j++)
                w[i] = (w[i] << 8) | (pb[i * 8 + j] & 0xFF);
        System.out.println("sha-512 preprocessed words (W0-W15):");
        for (int i = 0; i < 16; i++)
            System.out.printf("W%-2d = %016X%n", i, w[i]);
        Socket s = new Socket("localhost", 5000);
        DataOutputStream out = new DataOutputStream(s.getOutputStream());
```

```
        for (int i = 0; i < 16; i++)
            out.writeLong(w[i]);
        out.flush(); out.close(); s.close();
        System.out.println("words sent to server.");
    }
}
```

}

```
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab> cd "C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto la
b 7"
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7> javac SHA512Socket.java
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7> java SHA512Socket
enter message: messi is the greatest of all time in football
binary representation:
01101101 01100101 01110011 01110011 01101001 00100000 01101001 01110011 00100000 01110100 01101000 01100101 00100000 01100111 0111001
0 01100101 01100001 01110100 01100101 01110011 01110100 00100000 01101111 01100110 00100000 01100001 01101100 01101100 00100000 01110
100 01101001 01101101 01100101 00100000 01101001 01101110 00100000 01100110 01101111 01101111 01110100 01100010 01100001 01101100 011
01100
sha-512 preprocessed words (W0-W15):
W0  = 6D65737369206973
W1  = 2074686520677265
W2  = 6174657374206F66
W3  = 20616C6C2074696D
W4  = 6520696E20666F6F
W5  = 7462616C6C800000
W6  = 0000000000000000
W7  = 0000000000000000
W8  = 0000000000000000
W9  = 0000000000000000
W10 = 0000000000000000
W11 = 0000000000000000
W12 = 0000000000000000
W13 = 0000000000000000
W14 = 0000000000000000
W15 = 0000000000000168
words sent to server.
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7>
```

```
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab> cd "C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto la
b 7"
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7> javac SHA512Socket.java
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7> java SHA512Socket s
server waiting...
received message schedule words:
W0  = 6D65737369206973
W1  = 2074686520677265
W2  = 6174657374206F66
W3  = 20616C6C2074696D
W4  = 6520696E20666F6F
W5  = 7462616C6C800000
W6  = 0000000000000000
W7  = 0000000000000000
W8  = 0000000000000000
W9  = 0000000000000000
W10 = 0000000000000000
W11 = 0000000000000000
W12 = 0000000000000000
W13 = 0000000000000000
W14 = 0000000000000000
W15 = 0000000000000168
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7>
```

Qn.) A financial institution is implementing the Digital Signature Standard (DSS) to ensure the authenticity and integrity of electronic transactions between its clients and the bank's secure server. Alice, a customer, needs to submit a high-value transaction request to the bank, and the bank must verify that the request was genuinely sent by Alice and was not altered during transmission. Alice generates a digital signature using her private key and attaches it to the transaction request before sending it over a TCP socket to the bank's server. Upon receiving the request, the bank verifies the signature using Alice's public key to confirm its authenticity. Implement the DSS using socket programming to securely sign and verify transaction requests. Compute the digital signature for any given transaction using the signing algorithm and transmit it over a network. Implement the verification process where the bank validates the signature using Alice's public key before processing the transaction. Demonstrate how DSS ensures non-repudiation, prevents forgery, and protects against message tampering. Explain the role of cryptographic hash functions, private keys, and public keys in maintaining the security of digital signatures. The implementation must support any valid set of parameters and be tested using socket programming in a language of your choice, such as Python, Java, or C.

Digital Signature Standard (DSS) using Socket Programming

Objective

To implement the Digital Signature Standard (DSS) using socket programming so that a transaction sent by a client can be authenticated and verified by a bank server.

Overview

Digital signatures are used to ensure authentication, integrity, and non-repudiation in electronic communication. In this implementation, Alice (client) signs a transaction request using her private key and sends it to the bank server through a TCP socket connection. The bank verifies the signature using Alice's public key before processing the transaction.

The program uses:

- DSA (Digital Signature Algorithm) for digital signatures
- SHA-256 for hashing the message
- Java Socket Programming for communication between client and server.

Working Procedure

1. Key Generation
    - Alice generates a DSA key pair consisting of a private key and a public key.

    - The private key is used for signing the transaction and the public key is shared with the server for verification.

2. Transaction Creation
    - Alice enters a transaction message (for example: money transfer request).
3. Hash Generation
    - The transaction message is hashed using SHA-256 to produce a fixed-length digest that represents the message.
4. Digital Signature Generation
    - Alice signs the message using the private key and the SHA256withDSA algorithm.
    - This generates a digital signature.
5. Transmission using Socket
    - The client sends the transaction message, digital signature, and public key to the bank server using a TCP socket connection.
6. Signature Verification
    - The bank server receives the data and verifies the signature using Alice's public key.
    - If verification succeeds, the transaction is accepted; otherwise, it is rejected.

Security Features:

- Authentication: Confirms that the transaction was sent by Alice.
- Integrity: Any change to the message will cause signature verification to fail.
- Non-repudiation: Alice cannot deny sending the transaction since it was signed with her private key.
- Forgery Prevention: Without the private key, an attacker cannot generate a valid signature.

Conclusion:

The implementation demonstrates how Digital Signature Standard (DSS) secures electronic transactions. By using hash functions, private key signing, and public key verification, the system ensures that transactions transmitted over the network are authentic and unaltered.

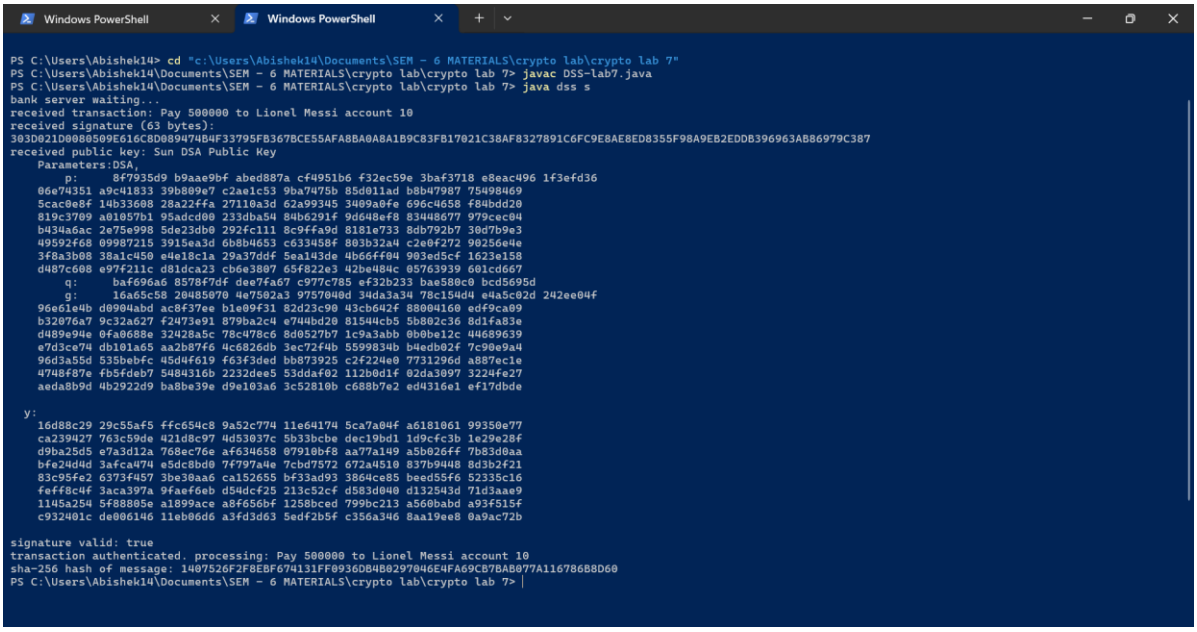
Code:

```
import java.io.*;
import java.net.*;
import java.security.*;
import java.util.*;

class dss {
    public static void main(String[] a) throws Exception {
        if (a.length > 0 && a[0].equals("s")) {
            ServerSocket ss = new ServerSocket(6000);
            System.out.println("bank server waiting...");
            Socket s = ss.accept();
            ObjectInputStream in = new ObjectInputStream(s.getInputStream());
            String msg = (String) in.readObject();
            byte[] sig = (byte[]) in.readObject();
            PublicKey pk = (PublicKey) in.readObject();
            System.out.println("received transaction: " + msg);
            System.out.println("received signature (" + sig.length + " bytes):");
            StringBuilder sb = new StringBuilder();
            for (byte b : sig) sb.append(String.format("%02X", b));
            System.out.println(sb);
            System.out.println("received public key: " + pk);
            Signature v = Signature.getInstance("SHA256withDSA");
            v.initVerify(pk);
            v.update(msg.getBytes("UTF-8"));
            boolean ok = v.verify(sig);
            System.out.println("signature valid: " + ok);
```

```
    if (ok)
        System.out.println("transaction authenticated. processing: " + msg);
    else
        System.out.println("signature invalid! transaction rejected.");
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] h = md.digest(msg.getBytes("UTF-8"));
    System.out.print("sha-256 hash of message: ");
    for (byte b : h) System.out.printf("%02X", b);
    System.out.println();
    in.close(); s.close(); ss.close();
} else {
    KeyPairGenerator kg = KeyPairGenerator.getInstance("DSA");
    kg.initialize(2048);
    KeyPair kp = kg.generateKeyPair();
    PrivateKey sk = kp.getPrivate();
    PublicKey pk = kp.getPublic();
    System.out.println("alice's key pair generated (DSA 2048-bit)");
    System.out.println("private key: " + sk);
    System.out.println("public key: " + pk);
    Scanner sc = new Scanner(System.in);
    System.out.print("enter transaction: ");
    String msg = sc.nextLine();
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] h = md.digest(msg.getBytes("UTF-8"));
    System.out.print("sha-256 hash: ");
    for (byte b : h) System.out.printf("%02X", b);
    System.out.println();
```

```
        Signature sg = Signature.getInstance("SHA256withDSA");
        sg.initSign(sk);
        sg.update(msg.getBytes("UTF-8"));
        byte[] sig = sg.sign();
        System.out.println("digital signature (" + sig.length + " bytes):");
        StringBuilder sb = new StringBuilder();
        for (byte b : sig) sb.append(String.format("%02X", b));
        System.out.println(sb);
        Socket s = new Socket("localhost", 6000);
        ObjectOutputStream out = new ObjectOutputStream(s.getOutputStream());
        out.writeObject(msg);
        out.writeObject(sig);
        out.writeObject(pk);
        out.flush();
        System.out.println("transaction and signature sent to bank server.");
        out.close(); s.close();
    }
  }
}
```



Windows PowerShell | Windows PowerShell

```
PS C:\Users\Abishek14> cd "c:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7"
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7> javac DSS-lab7.java
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7> java dss
alice's key pair generated (DSA 2048-bit)
private key: sun.security.provider.DSAPrivateKey@987ea82b
public key: Sun DSA Public Key
    Parameters:
    p:
    8f7935d9 b9aae9bf abed887a cf4951b6 f32ec59e 3baf3718 e8eac496 1f3efd36
    06e74351 a9c41833 39b809e7 c2ae1c53 9ba7475b 85d011ad b8b47987 75498469
    5cac0e8f 14b33608 28a22ffa 27110a3d 62a99345 3409a0fe 696c4658 f84bdd20
    819c3709 a01057b1 95adcd00 233dba54 84b6291f 9d648ef8 834486677 979cec04
    b434a6ac 2e75e998 5de23db0 292fc111 8c9ffa9d 8181e733 8db792b7 30d7b9e3
    49592f68 09987215 3915ea3d 6b8b4653 c633458f 803b32a4 c2e0f272 90256e4e
    3f8a3b08 38a1c450 e4e18c1a 29a37ddf 5ea143de 4b66ff04 903ed5cf 1623e158
    d487c608 e97f211c d01dca23 cb6e3807 65f022e3 42be484c 05763939 601cd667
    q:
    baf696a6 8578f7df dee7fa67 c977c785 ef32b233 bae580c0 bcd5695d
    g:
    16a65c58 20485070 4e7502a3 9757040d 34da3a34 78c154d4 e4a5c02d 242ee04f
    96e61e4b d0904abd ac8f37ee b1e09f31 82d23c90 43cb642f 88004160 edf9ca09
    b32076a7 9c32a627 f2473e91 879ba2c4 e744bd20 81544cb5 5b802c36 8d1fa83e
    d489e94e 0fa0688e 32428a5c 78c478c6 8d0527b7 1c9a3abb 0b0be12c 44689639
    e7d3ce74 db101a65 aa2b87f6 4c6826db 3ec72f4b 5599834b b4edb02f 7c90e9a4
    96d3a55d 535bebfc 45d4f619 f63f3ded bb873925 c2f224e0 7731296d a887ec1e
    4748f87e fb5fdeb7 5484316b 2232dee5 53ddaf02 112b0d1f 02da3097 3224fe27
    aeda8b9d 4b2922d9 ba8be39e d9e103a6 3c52810b c688b7e2 ed4316e1 ef17dbde

  y:
    16d88c29 29c55af5 ffc654c8 9a52c774 11e64174 5ca7a04f a6181061 99350e77
    ca239427 763c59de 421d8c97 4d53037c 5b33bcbe dec19bd1 1d9cfc3b 1e29e28f
    d9ba25d5 e7a3d12a 768ec76e af634658 07910bf8 aa77a149 a5b026ff 7b83d0aa
    bfe24d4d 3afca474 e5dc8bd0 7f797a4e 7cbd7572 672a4510 837b9448 8d3b2f21
    83c95fe2 6373f457 3be38aa6 ca152655 bf33ad93 3864ce85 beed55f6 52335c16
    feff8c4f 3aca397a 9faef6eb d54dcf25 213c52cf d583d040 d132543d 71d3aae9
    1145a254 5f888085e a1899ace a8f656bf 1258bced 799bc213 a560babd a93f515f
    c932401c de006146 11eb06d6 a3fd3d63 5edf2b5f c356a346 8aa19ee8 0a9ac72b

enter transaction: Pay 500000 to Lionel Messi account 10
sha-256 hash: 1407526F2F8EBF674131FF0936DB4B0297046E4FA69CB7BAB077A116786B8D60
digital signature (63 bytes):
303D021D0080509E616C8D089474B4F33795FB367BCE55AFA8BA0A8A1B9C83FB17021C38AF8327891C6FC9E8AE8ED8355F98A9EB2EDDB396963AB86979C387
transaction and signature sent to bank server.
PS C:\Users\Abishek14\Documents\SEM - 6 MATERIALS\crypto lab\crypto lab 7>
```